Chapter 9

Immediate Communication with ActionCable

In the last chapter, we talked about communicating with the server using regular HTTP requests and receiving either HTML or JSON in response. One feature of our server interactions in both Stimulus and React is that we are simulating a real-time connection with the server by polling the server repeatedly. Polling this way has a couple of potential drawbacks. There's some extra overhead to make all the HTTP calls locally, and pestering the server often can have bad performance implications.

An alternative for client-server communications is to use the WebSocket protocol. The WebSocket protocol allows a client and a server to communicate over a single long-lived connection. A common analogy is that regular HTTP connections perform like walkie-talkies in that only one of the two parties can broadcast at a time. A WebSocket is like a phone, where both parties can talk at once. A WebSocket is effectively a stream in both directions, where both client and server can perform actions when new data is received.

Rails provides a library called ActionCable,[67] which is a wrapper around WebSockets that makes it easy to use a WebSocket on both the client and the server end. Beyond that, Hotwire and Turbo provide utilities that allow us to combine ActionCable and Turbo Streams to provide ActionCable functionality without writing any new JavaScript.

In this chapter, we're going to take a look at the Turbo Streams methods that